How This Project Actually Unfolded (Informal Timeline)

Quick Version

If I strip this down to the real sequence: you did not jump straight into coding. You first set up a process for how work would happen (context tracking, prompts, gates, milestones), then built the app foundation, then layered ligand workflows, then FragMap workflows, then overview content, then submission packaging and polish.

So the project was run as **process first -> foundation first -> features in slices -> validation every step -> submission packaging**.

The Order You Executed Things

1) You framed the assignment and constraints first

Before implementation, you anchored the project to the SILCS exercise requirements and committed to a milestone-based delivery model. That gave you a clear scope boundary: browser-only demo, no backend, required routes, required ligand/FragMap interactions, and clear acceptance criteria.

2) You decided the stack and app shape early

From repo state and early implementation direction, you locked into:

- Vue 2 + TypeScript + Vuetify + Vuex + Vue Router
- NGL as the molecular viewer
- Static/client-side asset flow with staging and manifest-driven runtime loading

This matters because once this was set, every later design/validation decision was made around this architecture.

3) You created the execution system before heavy feature coding (2026-02-15)

This is the part people usually skip, but you did it early:

- Added/standardized durable context memory files (`docs/context/current-state.md`, `docs/context/decision-log.md`, `docs/context/next-agent-brief.md`)
- Added handoff template/process
- Enforced startup read order for agents
- Standardized milestone prompts and review flow

In plain terms: you built a mini operating system for the project, not just app code.

4) You moved to a two-step UI workflow: Prompt A then Prompt B

For user-facing work, the order became:

1. Prompt A: design previews only
2. Wait for explicit `APPROVED UI PREVIEW`
3. Prompt B: implementation

This controlled churn. Instead of rewriting UI after coding, you previewed first and coded second.

5) You implemented and stabilized the core foundation (M1 -> M3)

M1: scaffold + routing

- Route contracts and shell wiring (`/` and `/viewer`)
- Base app nav and route components

M2: data manifest + startup validation

- Asset staging and generated manifest
- Runtime startup validation and disable intent behavior

M3: viewer core lifecycle

- Viewer shell layout
- NGL stage init/ready/resize/destroy
- Default state/camera behavior
- Loading/fallback/error handling

You validated each stage before moving on.

6) You implemented ligand workflows next, in phases (M4)

M4A first (core pose workflow)

- Baseline/refined controls
- 4 pose states (baseline-only, refined-only, both-visible, both-hidden)
- Per-pose failure handling

M4B second (featured ligand switching)

- In-place switching across approved featured subset
- Camera preservation and switch stability

M4C deferred

- Full searchable ligand list intentionally deferred as non-blocking

This was a deliberate sequencing choice: deliver critical interaction quality first, defer bigger list/search scope.

7) You broke FragMap work into fine-grained slices (M5), then executed in that order

You explicitly ran M5 as a chain, not a blob:

- M5.1 shell/tabs
- M5.2 primary map toggles + lazy load/cache behavior
- M5.2a wireframe rendering contract
- M5.2b protein visibility toggle placement/behavior
- M5.3 advanced rows + exclusion behavior
- M5.4 per-map iso controls
- M5.5 bulk actions
- M5.5a reset semantics refinement
- M5.6 reliability hardening (retry + stale-intent guards)

Optional M5.2c parity investigation was inserted as exploratory and later de-scoped as non-blocking.

This was the most iterative part of the project: preview revisions, implementation, validator updates, regression reruns, and bug fixes.

8) You fixed bugs while moving forward, not in a separate cleanup phase

As you progressed, you resolved issues in-line:

- Intermittent nav click interception from snackbars
- Timing-sensitive validator failures
- Exclusion-map visibility bug (fixed via exclusion-specific iso behavior)
- Camera regression after ligand switches/resets
- Validation harness collision issues from parallel execution

So your execution style was "advance one slice, fix regressions immediately, revalidate whole chain."

9) You implemented M6 (overview page) after the interactive core was stable

Once viewer interactions were solid, you finalized the narrative side:

- Home page scientific text
- CTA route behavior

- External reference links
- Copy refinements and spec/validator wording alignment

This gave you complete M1-M6 user-facing coverage before submission packaging.

10) You switched to submission mode and prioritized deliverables over unfinished milestones

With time constraints, you intentionally prioritized:

- Readme quality
- Requirement mapping clarity
- Validation command coverage listing
- Documented tradeoffs/limitations
- Live/repo links

And explicitly deferred:

- M7 instrumentation/evidence pack
- M8 final hardening/signoff

That was a practical project-management decision, not an oversight.

11) You did post-M6 polish passes

After core delivery, you still did UI polish:

- Viewer interaction hints
- Top bar simplification
- Moving reset control to better viewer placement

And revalidated key gates after those changes.

12) You produced reviewer-friendly reporting artifacts last (2026-02-27)

You then generated formal handoff/report artifacts in `docs/reports/` so the project journey is readable outside chat context.

Bottom Line: Your Real Execution Pattern

The project order was:

1. Understand assignment and constraints
2. Choose stack + architecture
3. Create planning/handoff/prompt process
4. Lock UI-gated workflow (Prompt A/Prompt B)
5. Build foundation (M1-M3)

6. Build ligand interactions (M4A/M4B)
7. Build FragMap interactions slice-by-slice (M5.x)
8. Build overview/narrative page (M6)
9. Package submission and polish UX
10. Export reports

This is a very deliberate "engineering-first" sequence: **control process early, reduce risk with slices, validate continuously, and package clearly for review.**

Where This Leaves You Right Now

Current state in that sequence:

- Completed and validated through M6
- M4C and M5.2c are intentionally non-blocking/deferred
- M7/M8 still pending if you want full AC evidence and hardening closure